IT4931 / IT5384 / IT4043E · SOICT · HUST

NoSQL Databases

Lecture 5 — NoSQL databases

Why NoSQL · Four models · CAP / BASE · 2026 outlook

Learning outcomes (CLO1)

1. **Explain** why specialized non-relational stores emerged beside RDBMS
2. **Match** access patterns to key-value, document, column-family, or graph
3. **Apply** CAP / BASE / eventual consistency when replicas and partitions appear
4. **Recognize** where vector search and SQL-over-stores fit in a 2026 data stack

Roadmap today

1. **5.1** Why NoSQL
2. **5.2** Data models — KV · document · CF · graph
3. **5.3** CAP · 2PC · BASE / eventual
4. **5.4** Vector databases
5. **5.5** SQL over stores

5.1 Why NoSQL

Why the landscape shifted

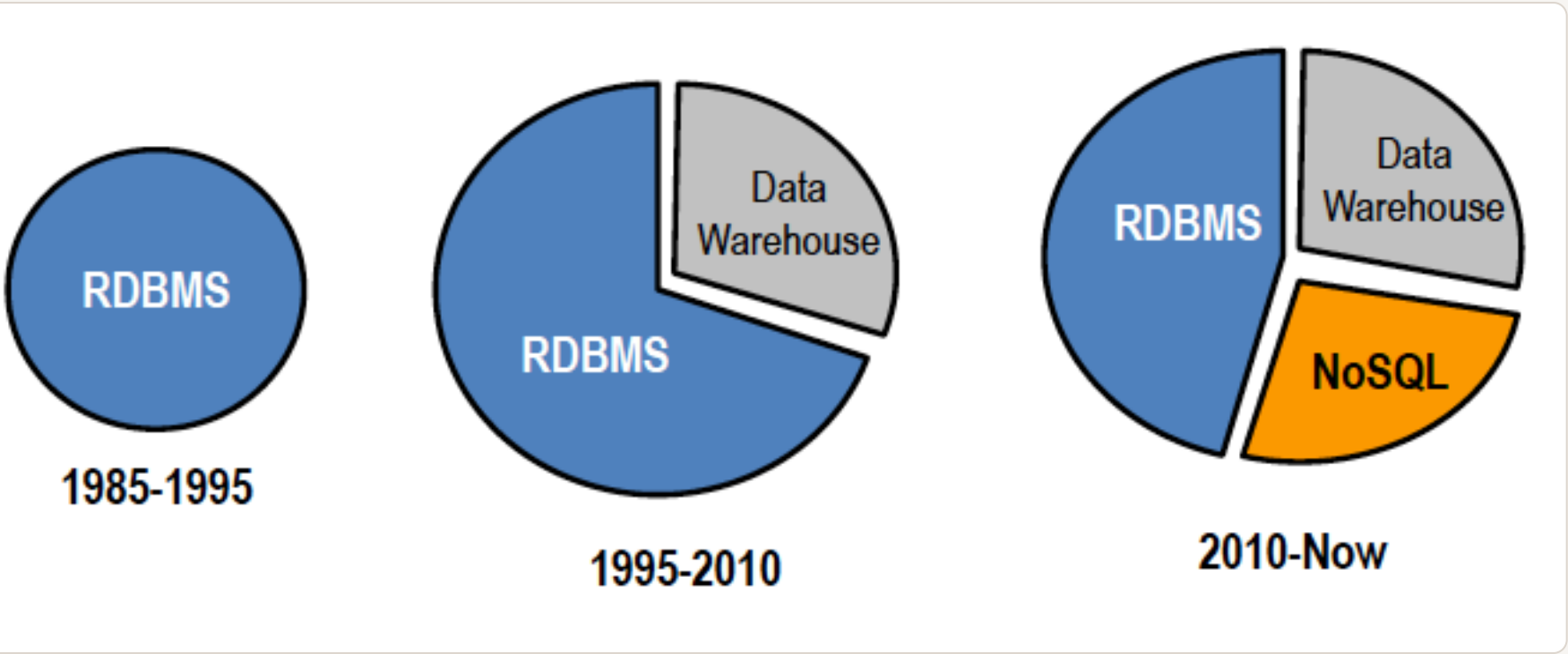
RDBMS remains central — but web/cloud scale forced **specialized** engines:

- **Scale-out** first (shard + replicate), not only bigger boxes
- **Flexible** / semi-structured data (logs, UGC, events)
- Not every workload needs full ACID + multi-table joins

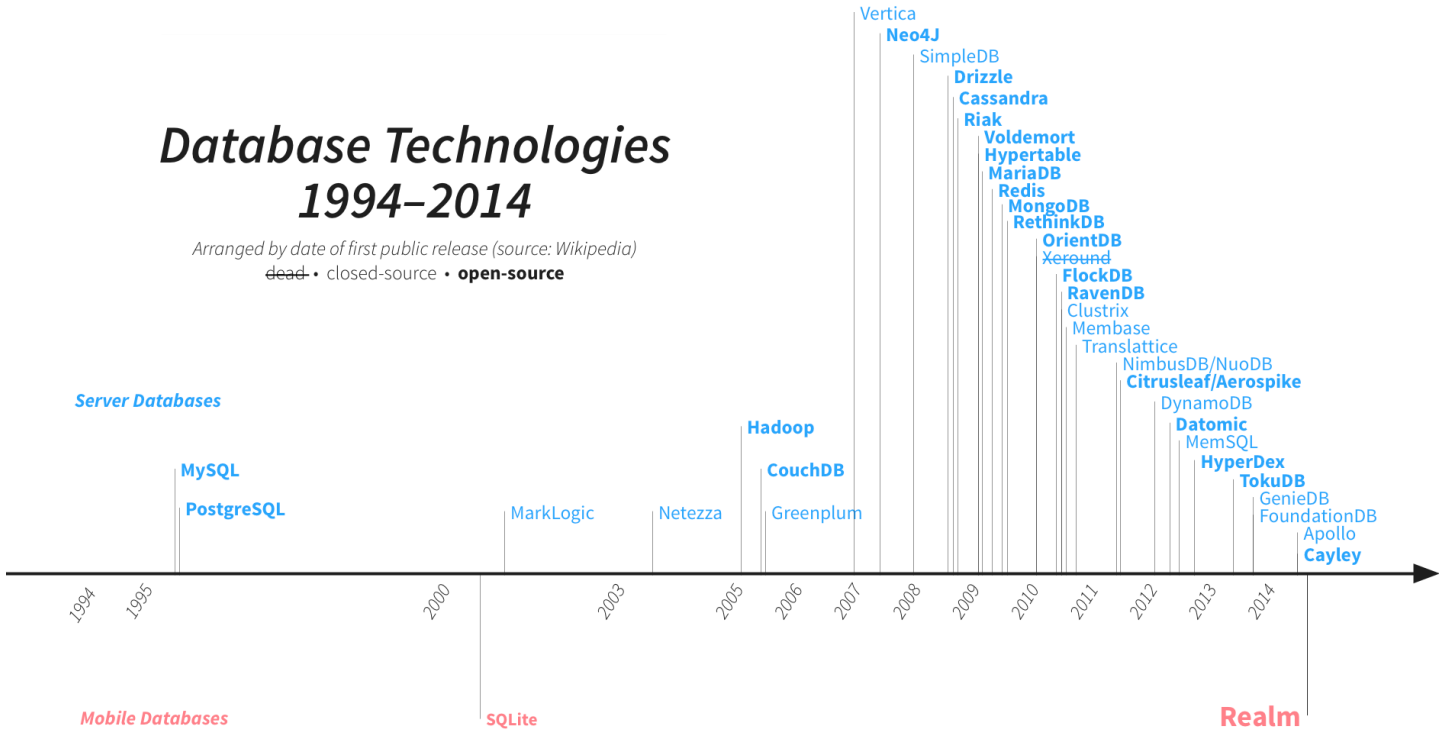
Next slides: eras → OLTP/OLAP context → “one size fits all?” → why NoSQL → then the four models.

Eras of databases

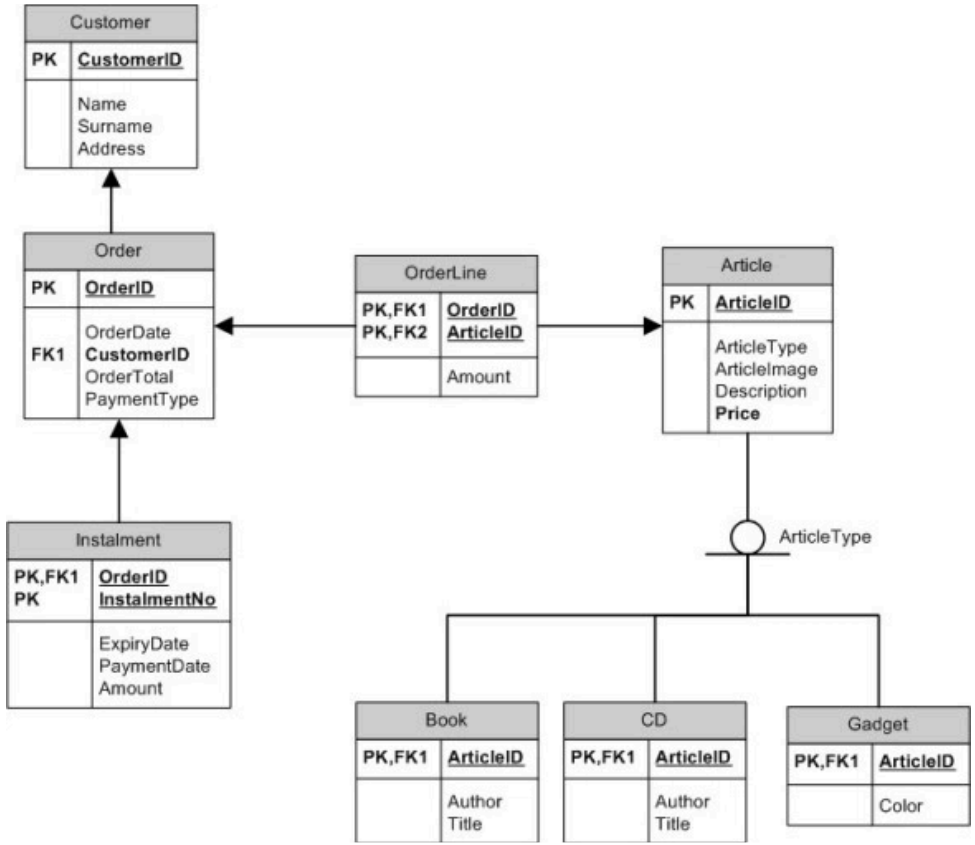
From navigational → relational → **polyglot / specialized** engines.



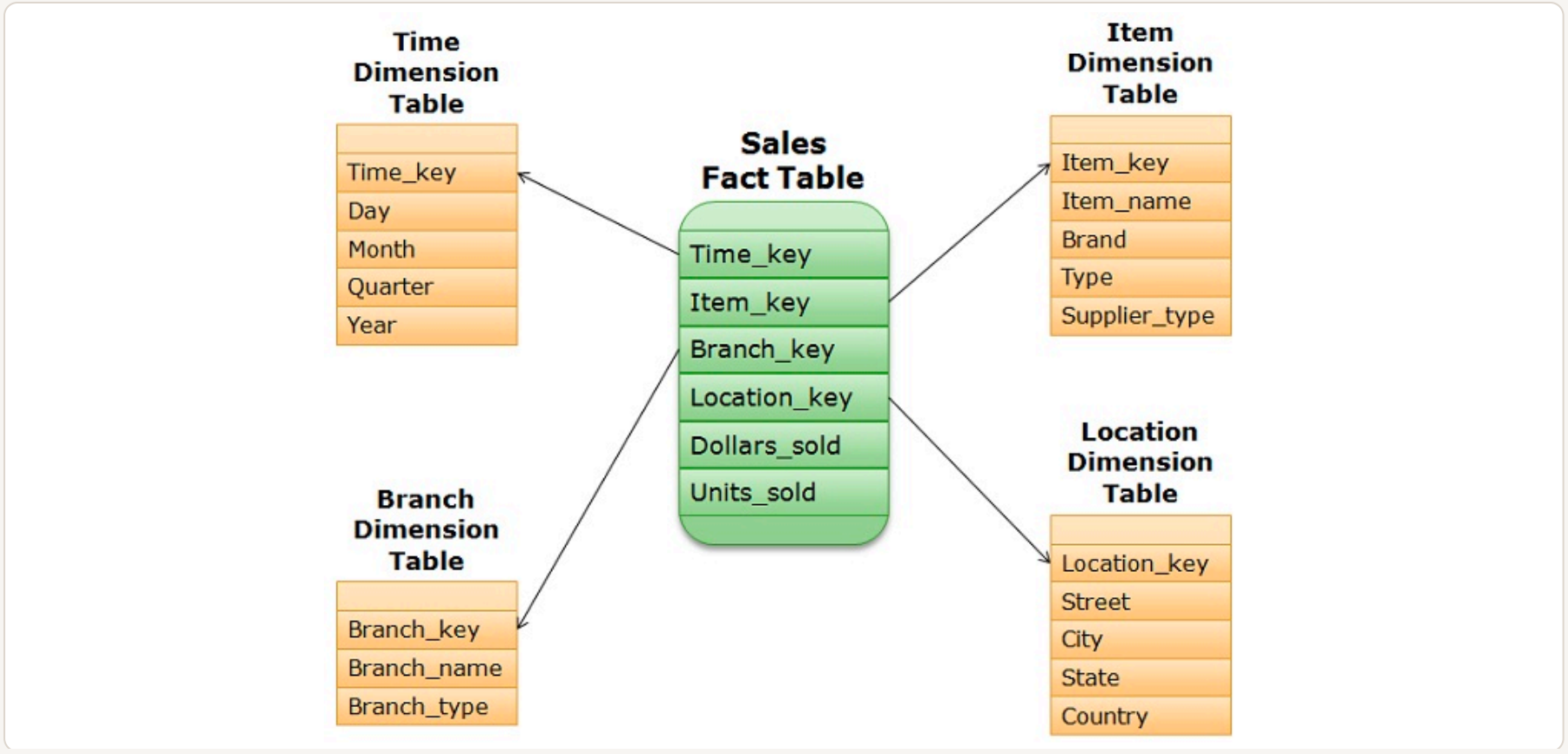
Eras of databases (detail)



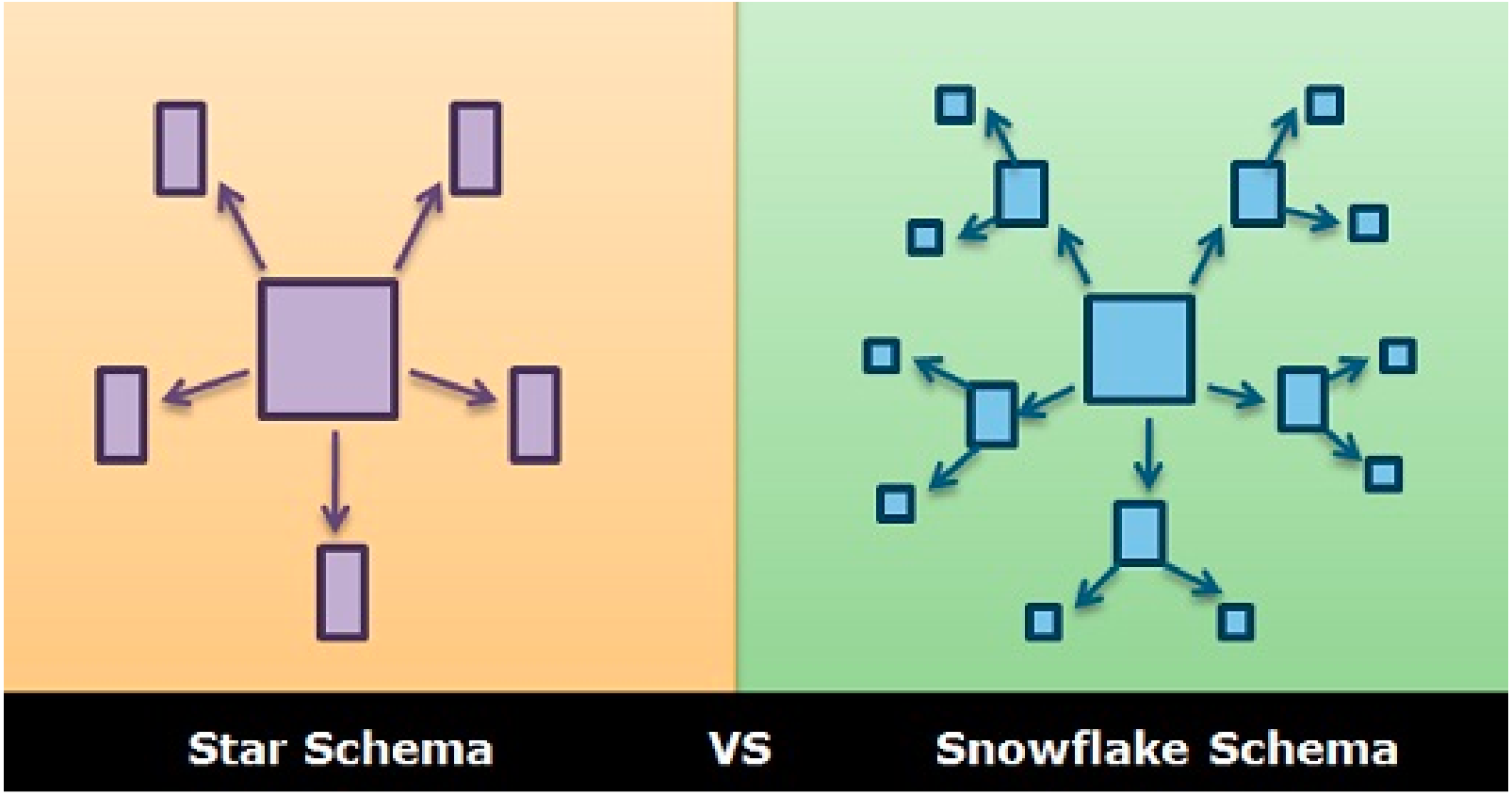
Before NoSQL — OLTP / OLAP / star schema



OLTP vs OLAP cube

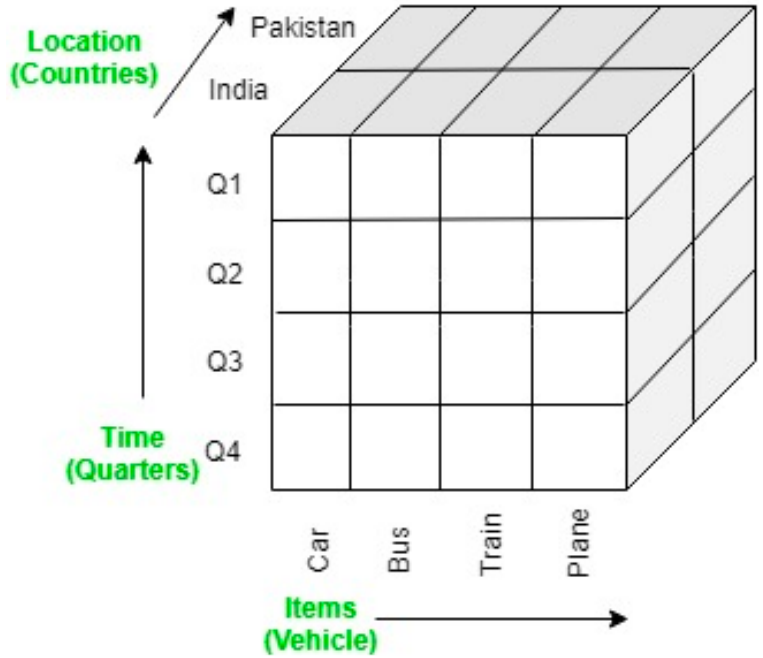


Star schema (warehouse classic)

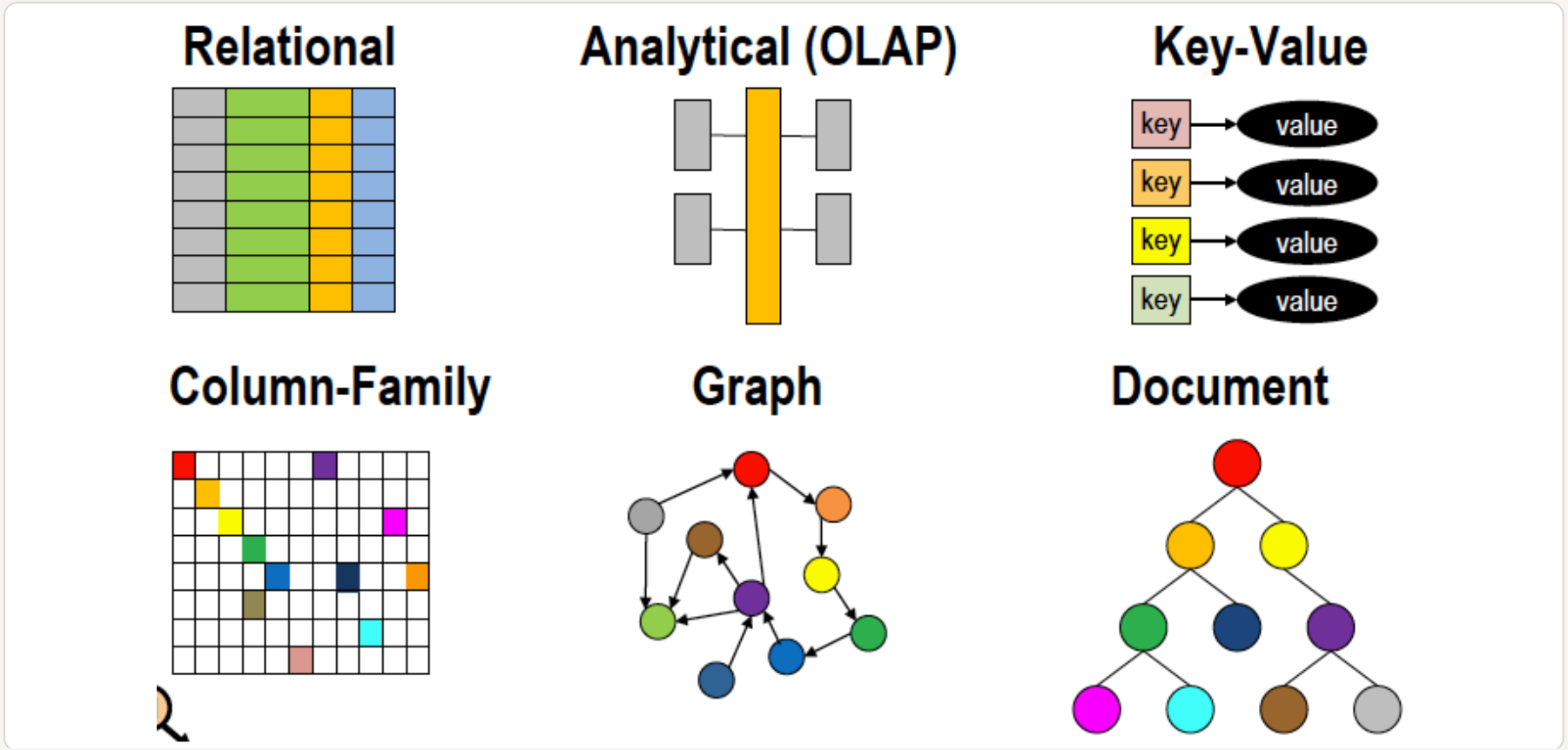


RDBMS — “one size fits all”?

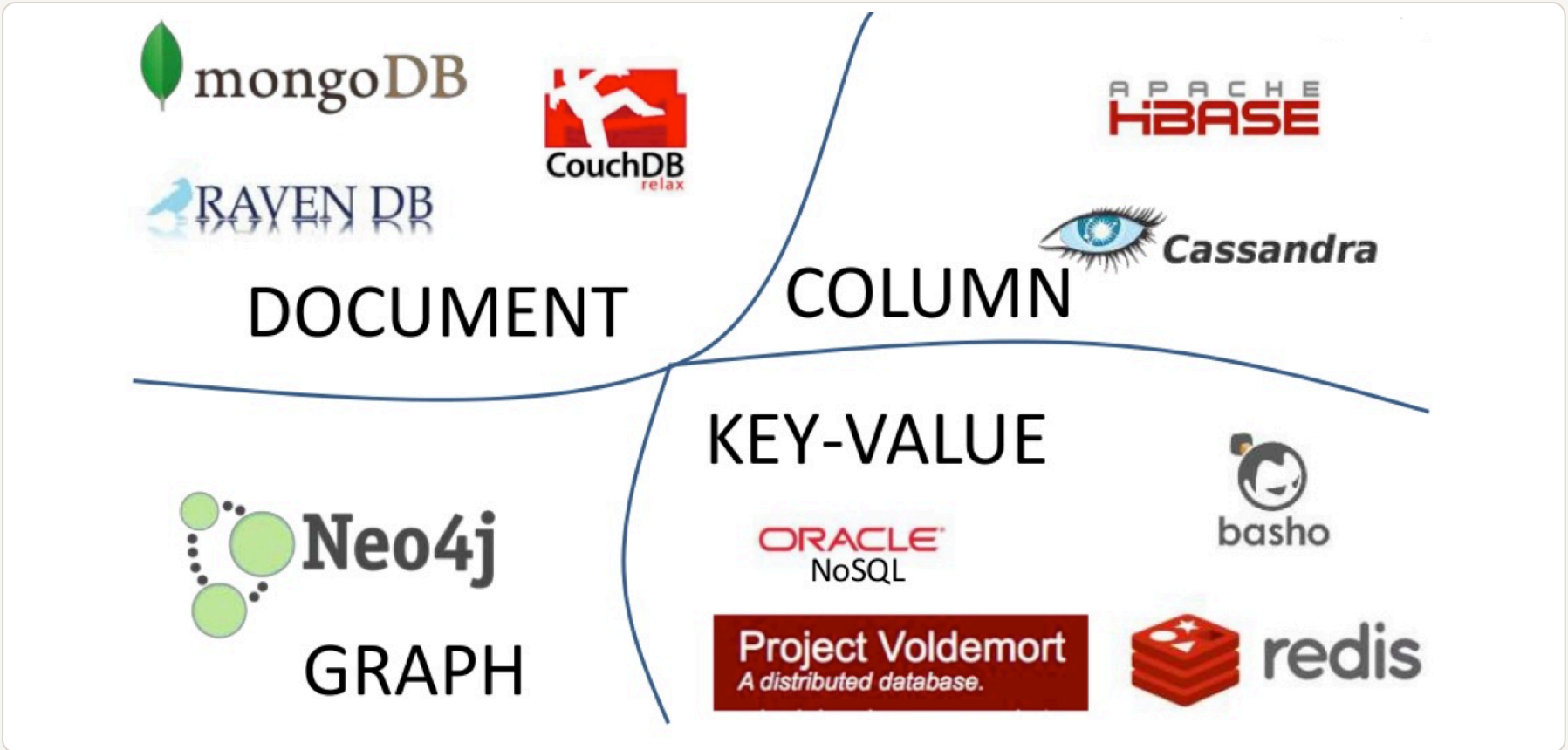
ICDE 2005 argument: commercial DBMS optimized for business transactions was pushed into **many** workloads → market **fractures** into specialized engines.



After NoSQL


Relational
Analytical (OLAP)
Key-Value
key
value
key
value
key
value
key
value
Column-Family
Graph
Document

NoSQL landscape



Why NoSQL?

Driver	Meaning
Horizontal scale	Many commodity nodes; partition / shard by design
Schema flexibility	Semi-structured / evolving data (logs, events, UGC)
Elasticity	Add/remove capacity; predictable low latency (some engines)
HA / geo	Replicate across racks / regions
Relaxed needs	Not every app needs full ACID + complex joins

*"NoSQL" ≈ **not only SQL** / non-relational trade-offs — not "never use SQL."*

SQL vs NoSQL (high level)

Aspect	Traditional SQL	NoSQL
Model	Tables, fixed schema	KV, document, CF, graph
Scale	Often vertical first	Horizontal first
Consistency	ACID common	Often BASE / eventual (tunable)
Query	Rich SQL + joins	Key/path/traversal; SQL via engines
Fit	Transactions, reporting	Scale-out, flexible schema, low latency

Use cases at scale (2026)

Classic (still common)	2026-heavy
User **profiles** / sessions	**RAG** & similarity search (vector DBs)
Caching & shopping carts	Real-time **personalization** / ranking
Product **catalogs** / CMS	**Feature stores** for ML serving
IoT / mobile event stores	Multi-region **low-latency** APIs
Fraud / messaging	Event-driven services on **Kafka + NoSQL**

Icons of "profile / cache / IoT / fraud …" still apply — today they sit next to AI retrieval and streaming platforms.

Popularity snapshot (DB-Engines, Sep 2026)

Rank	System	Model (primary)	Score
1	Oracle	Relational	1122
2	MySQL	Relational	845
3	Microsoft SQL Server	Relational	699
4	PostgreSQL	Relational	683
5	MongoDB	Document	381
6	Snowflake	Cloud warehouse	212
7	Databricks	Lakehouse / multi-model	168
8	Redis	Key-value	158
9	IBM Db2	Relational	111
10	SQLite	Embedded relational	97

Ranking

Also in the broader top tier for this course: **Elasticsearch**, **Cassandra**, **Neo4j**; vector stores (e.g.

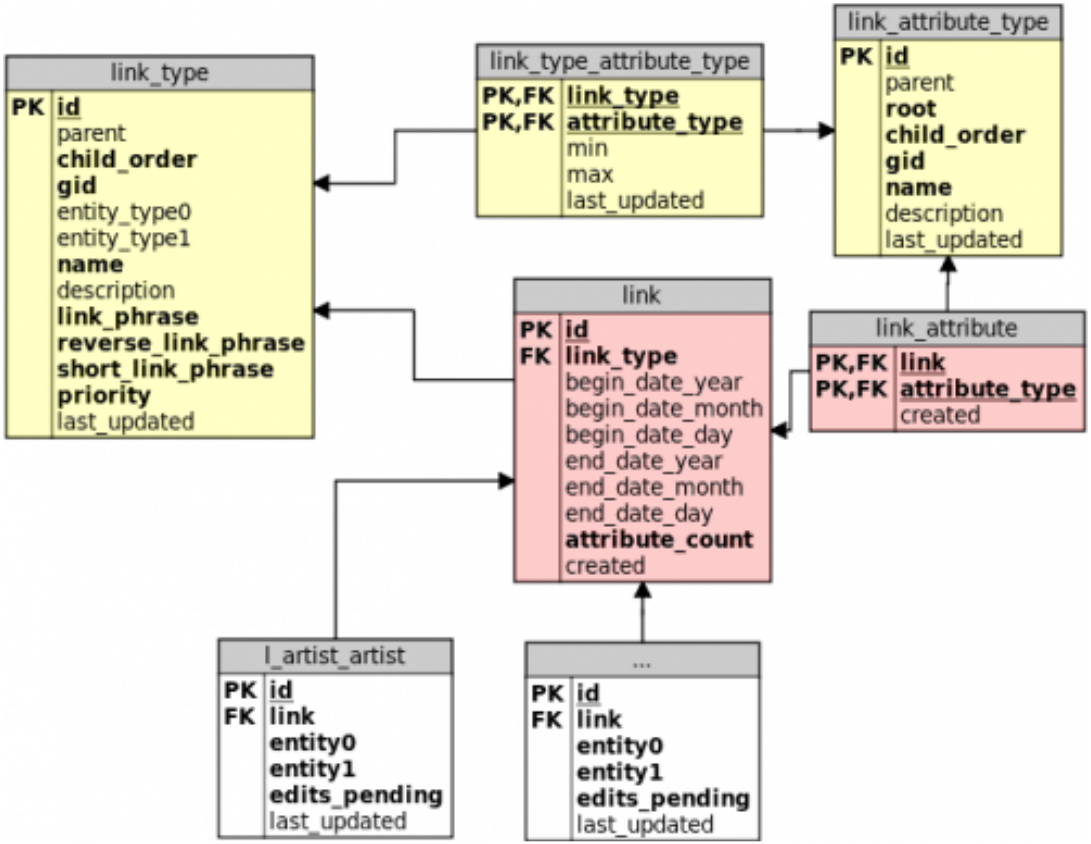
5.2 NoSQL data models

Four families (map)

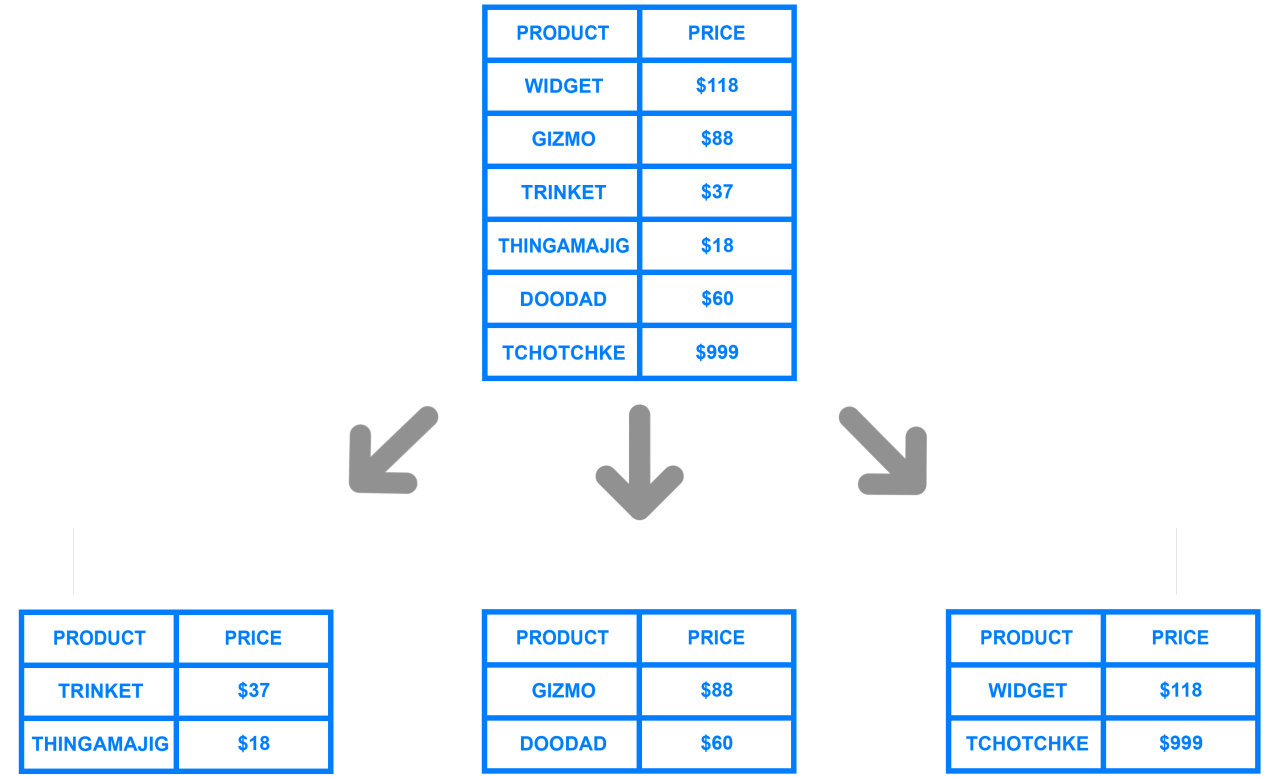
Model	Core abstraction	Typical engines
Key-value	`GET/PUT/DELETE` by key	Redis, Memcached, Riak
Document	JSON/BSON document + field indexes	MongoDB, CouchDB, DocumentDB
Column-family	`(row, family, column, ts) → cell`	Bigtable, HBase, Cassandra
Graph	Nodes + relationships + properties	Neo4j, Neptune

Same physical cluster may mix models (polyglot persistence). Next: each family in turn.

Relational model (recap)

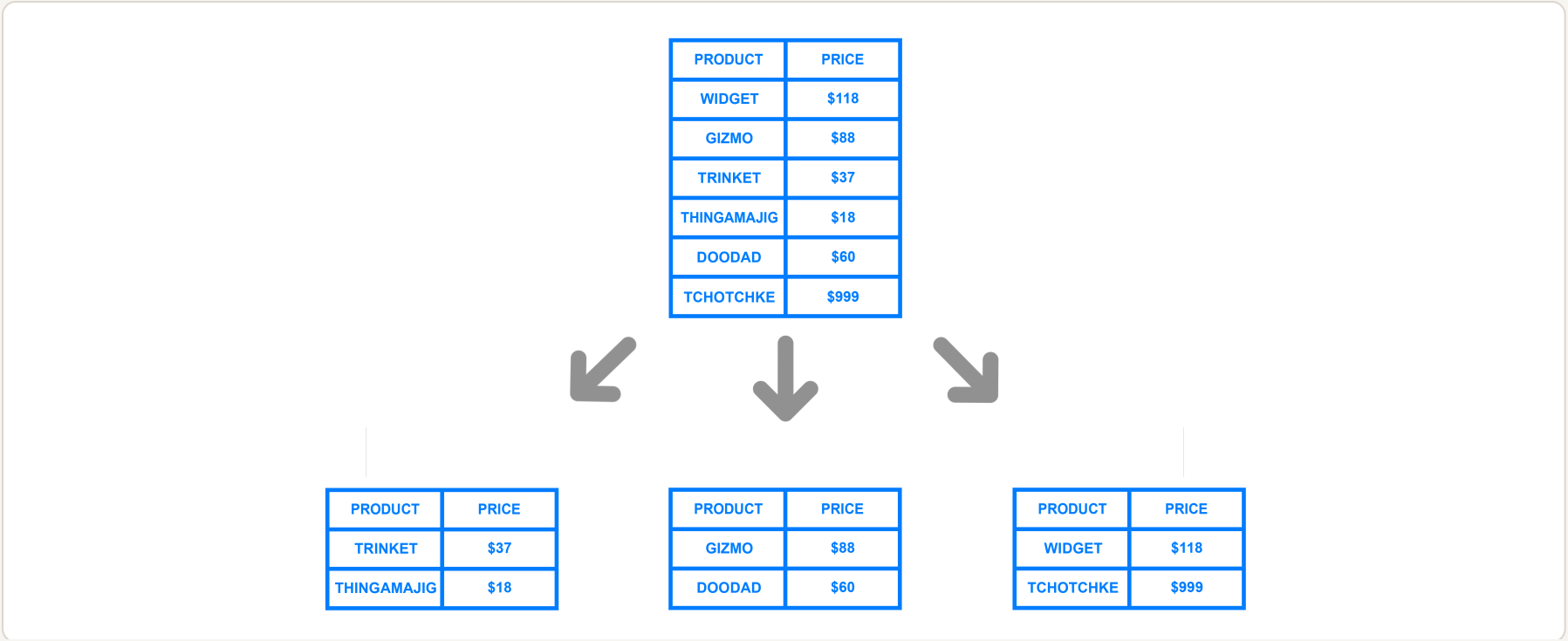


Key-value model



Key-value API

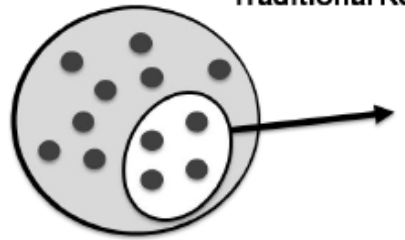
Simple interface: **GET / PUT / DELETE** — value opaque or structured; easy to shard by key.



Key-value vs table / relational

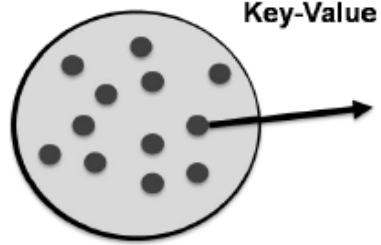


Traditional Relational Model



- Result set based on row values
- Value of rows for large data sets must be indexed
- Values of columns must all have the same data type

Key-Value Store Model



- All queries return a single item
- No indexes on values
- Values may contain any data type

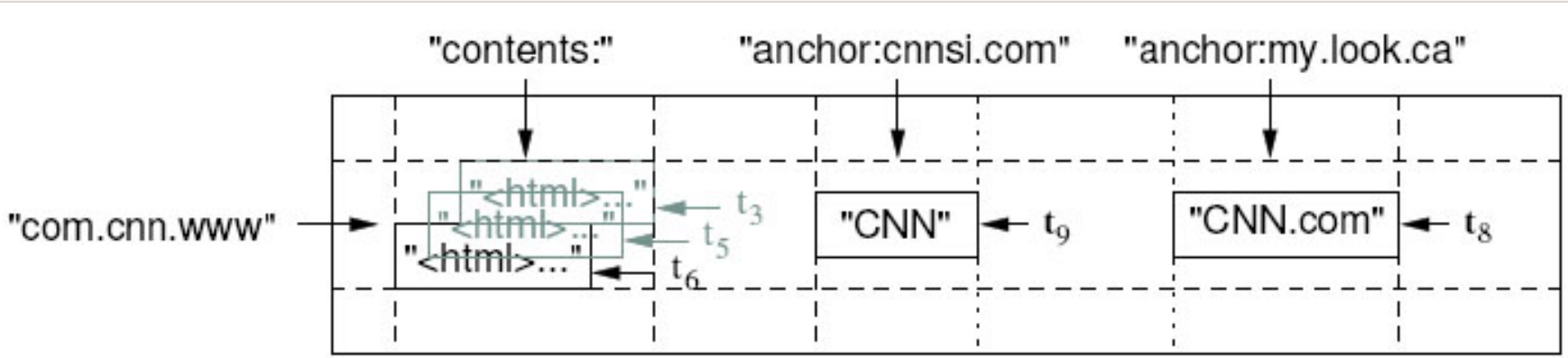
Memcached & Redis

	Memcached	Redis
Model	Simple **key → blob** cache	In-memory **data structures**
Persistence	Usually none (cache)	Optional AOF / RDB
Ops	get / set / delete	strings, lists, sets, hashes, sorted sets, pub/sub, …
Typical use	Hot-object cache in front of DB	Cache + sessions + queues + leaderboards

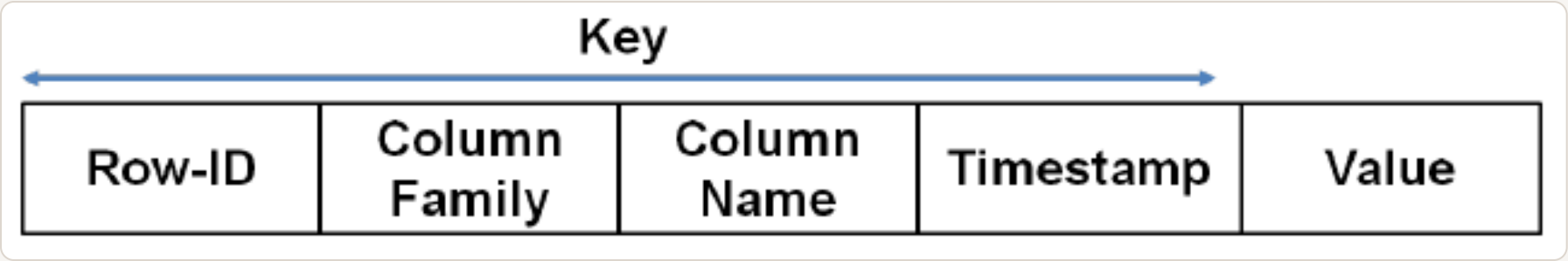
Both are **KV-class** serving stores — not a substitute for the system-of-record warehouse/lake.

Column-family store

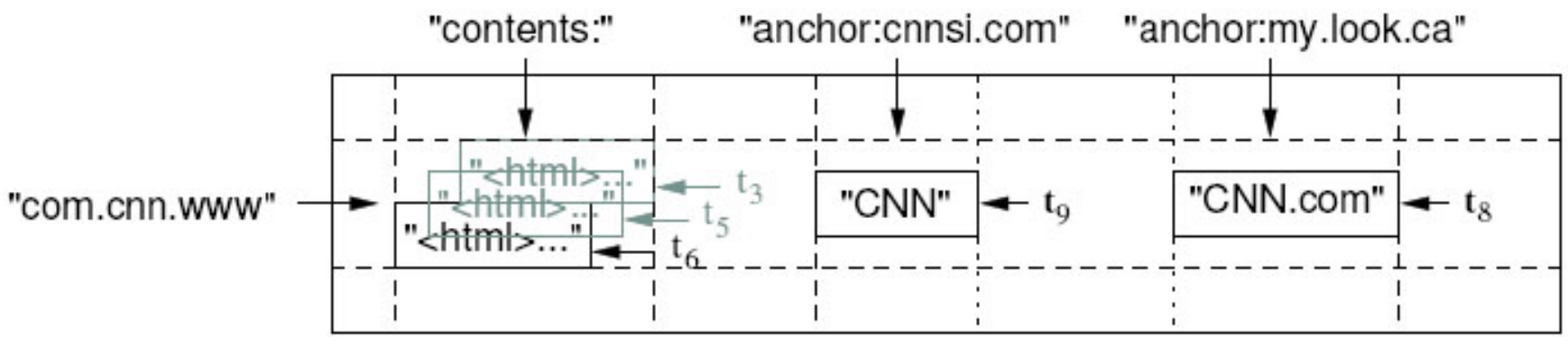
Sparse multi-dimensional map: **(row, column family, column, timestamp) → cell**.



Column families & sparse map



Bigtable-style sorted map



HBase & Cassandra (CF examples)

	HBase	Cassandra
Heritage	Bigtable on **HDFS**	Dynamo-inspired peer-to-peer
Topology	Master-ish regions + HDFS	Ring / gossip, no single master
Strength	Hadoop ecosystem · scans + random R/W	Write scale · multi-DC · tunable consistency
Course link	Seen with Hadoop (L02)	Consistency dials → **L06 DynamoDB**

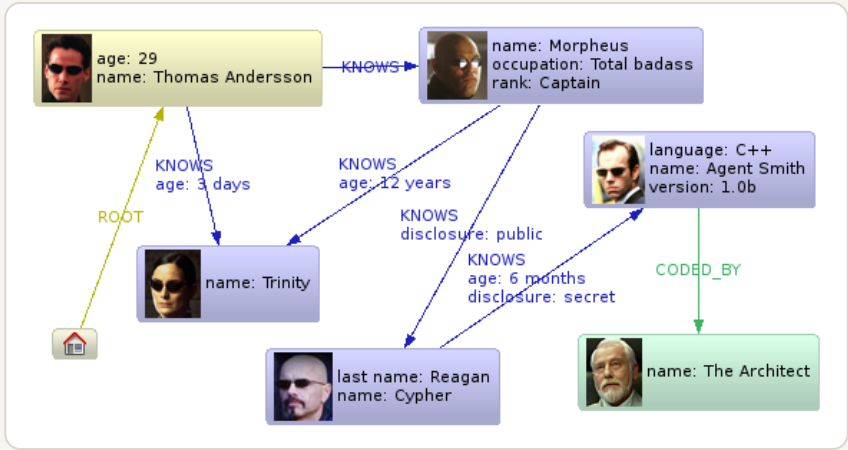
Pick by ops model: HDFS-coupled vs always-on wide-column cluster.

Graph model

Property graph = first-class connections

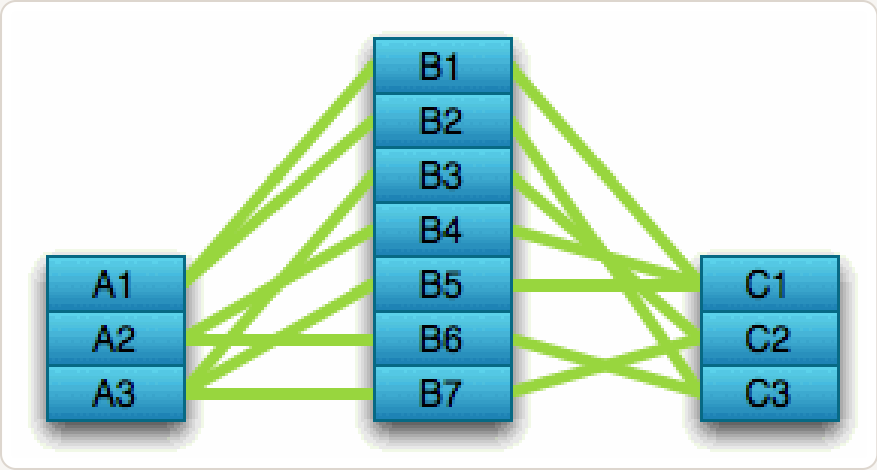
Abstraction	What it is
Node	Entity + properties
Relationship	Typed directed edge + properties
Property	Key–value on nodes *or* edges

Query = **traverse** edges — not JOIN by FK.

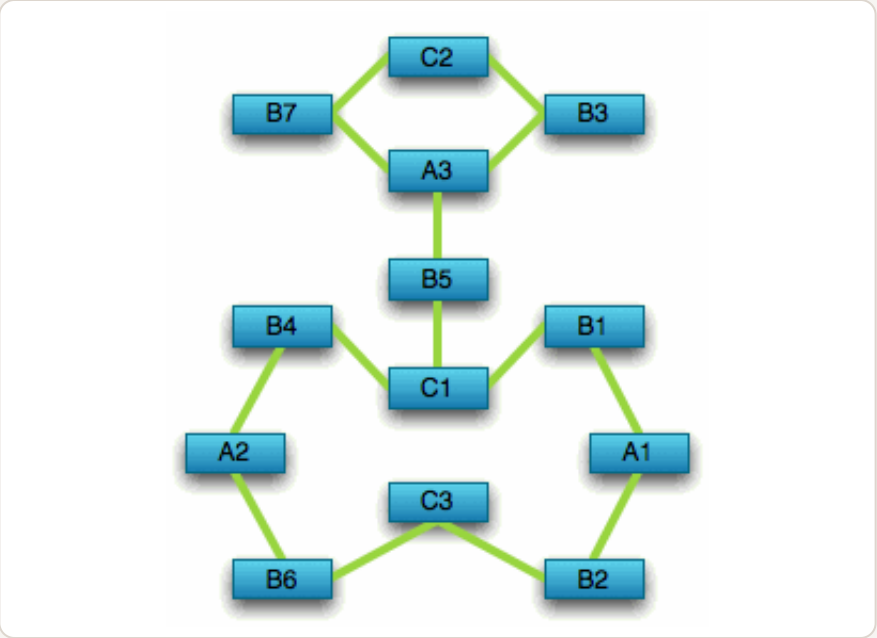


Graph vs other models

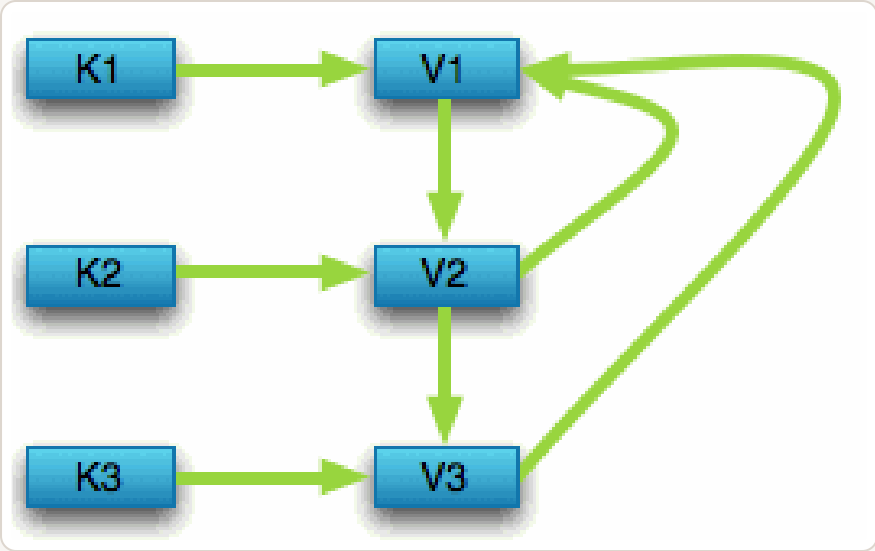
Relational — tables + joins



Native graph — direct adjacency



Graph vs document



Prefer graph when…	Prefer document when…
Multi-hop relationships	Nested JSON aggregates
Path / recommendation queries	Whole-document read/write
First-class edges	Schema-flexible records

Linked Open Data is a **web-scale** graph idea — same motivation, different stack.

Neo4j (example)

What	Property-graph DB (native graph storage + index-free adjacency)
Query	**Cypher** — pattern matching over nodes/relationships
Why here	Concrete product for the graph model (not the only one)
Fit	Fraud, recommendations, knowledge graphs, network ops

Contrast: RDBMS simulates graphs with JOIN tables; Neo4j stores edges as first-class.

Document store

```
{
  person: {
    first_name: "Peter",
    last_name: "Peterson",
    addresses: [
      {street: "123 Peter St"},
      {street: "504 Not Peter St"}
    ],
  }
}
```

Documents vs ORM complexity

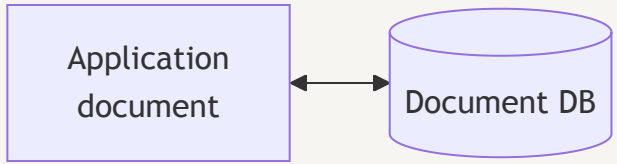
Relational apps often need an **object middle tier** (ORM) to shred / reassemble rows. Document stores keep the **same shape** in the app and in the DB.

Relational + ORM



T1–T4 = repeated impedance mismatch

Document mapping



No middle tier · no shredding · no reassembly

Document JSON shape

```
{
  person: {
    first_name: "Peter",
    last_name: "Peterson",
    addresses: [
      {street: "123 Peter St"},
      {street: "504 Not Peter St"}
    ],
  }
}
```

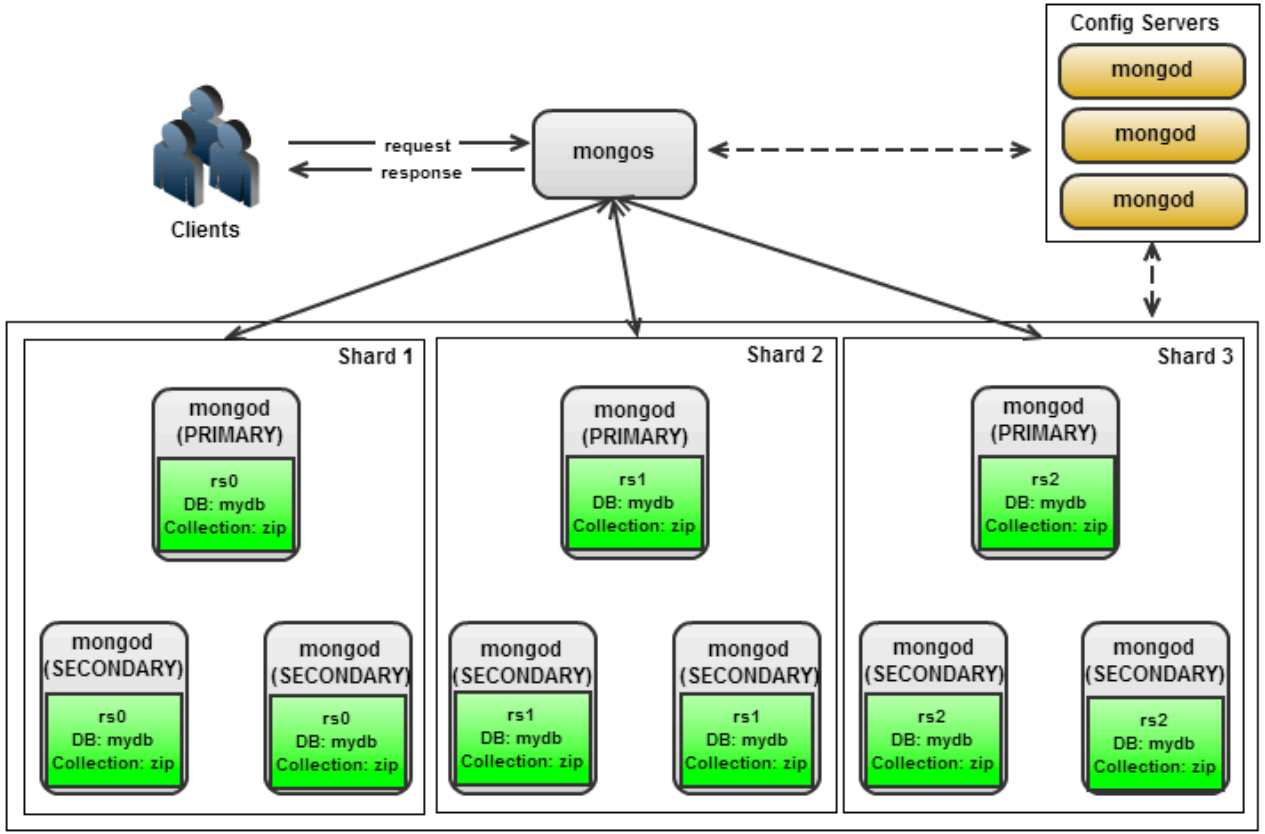
MongoDB

Open-source **JSON / BSON** document store (10gen / MongoDB Inc.).

Trait	What it means
Model	Documents in collections; flexible schema; field indexes
Scale-out	Built-in **sharding**; primary–secondary **replica sets**
API	Rich drivers (JS, Java, Python, C++, …); query language close to JSON
Fit	Web / mobile backends, catalogs, content, event-ish docs

Next slide: how replica sets + sharding compose a cluster.

MongoDB architecture — replica set & sharding



5.3 CAP · 2PC · eventual consistency

Why CAP appears here

Those NoSQL models **shard and replicate** by design. Shared data then forces three questions:

1. Do all clients see the **same** value after a write? (**C**)
2. Does every healthy node still **answer**? (**A**)
3. Does the cluster survive a **network split**? (**P**)

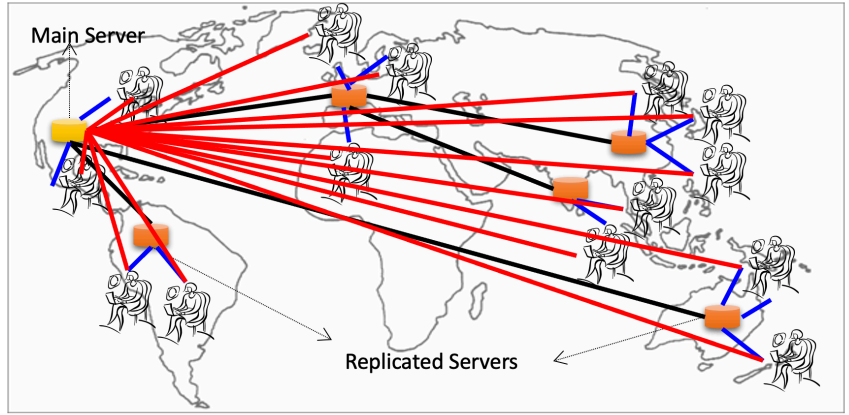
Next: CAP trade-off → why **2PC** is expensive → **BASE / eventual consistency**.

Scale-out: sharding & replication

Sharding — split the dataset across machines → parallel access.

Replication — keep copies → HA + read scale + geo locality.

Cost: every write may need to reach **several** replicas. Concurrent updates make **replica agreement** hard.

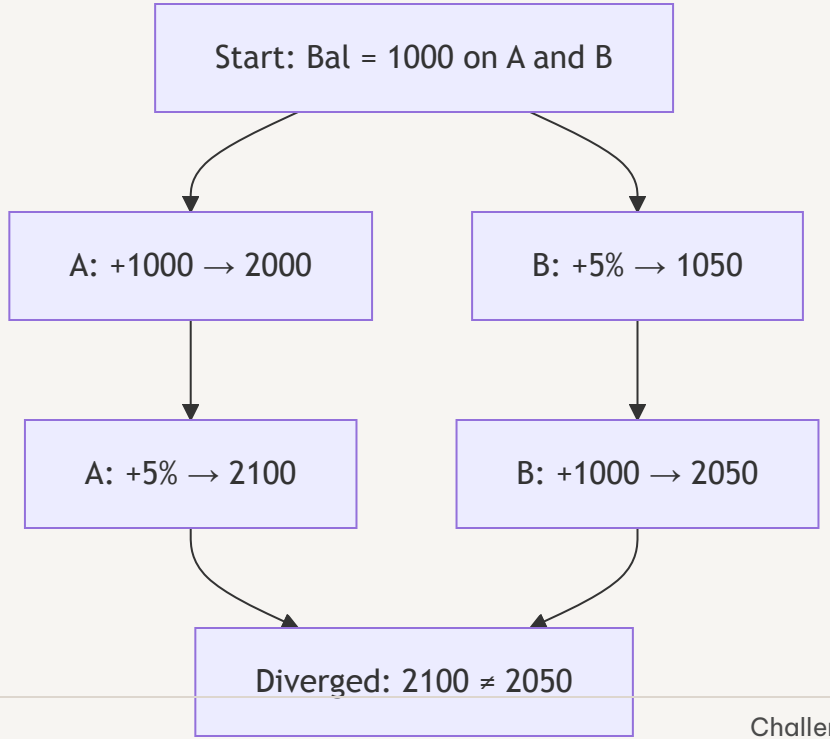


Consistency becomes hard

Bank DB replicated on **two servers**. Same start balance; two updates applied in **different order** → balances **diverge**.

Both start: `Bal = 1000`

	Event order	Result
Server A	① +$1000 → ② +5%	**2100**
Server B	① +5% → ② +$1000	**2050**



Three properties: C, A, P

Letter	Meaning (for shared data)	Failure mode if you give it up
C Consistency	After a successful write, **every** subsequent read (any replica) returns that write — or the system errors	Clients may read **stale** / conflicting values
A Availability	Every request to a non-failed node gets a **non-error** response (may be stale)	System may **refuse** / time out under stress
P Partition tolerance	Cluster keeps operating when the network **splits** nodes into groups	System assumes a reliable network (rare at scale)

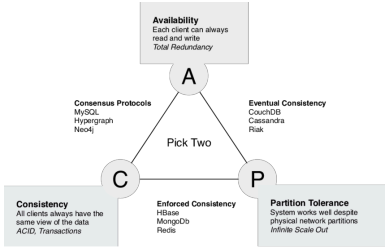
Brewer / CAP (PODC 2000): you cannot guarantee all three at once for a distributed shared-data store.

CAP theorem — the forced choice

In a real distributed system **P is not optional** (networks fail). Under a partition you choose:

Choice	Behavior when network splits	Typical goal
CP	Prefer **same data** → may reject / block requests on minority side	Strong consistency (bank-like)
AP	Prefer **answers** → both sides keep serving → replicas may diverge	Always-on UX (feeds, carts, CDN)

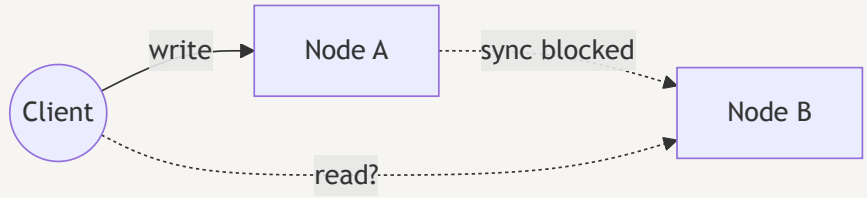
CA only "works" when there is effectively **no partition** (single node / tightly coupled cluster).



CAP — two-node thought experiment

1. Client writes to node **A**; strong **C** needs A → sync → **B**.
2. Network **partition** cuts A ↔ B.
3. Keep serving both sides → lose **C** (B may be stale).
4. Block until sync resumes → keep **C**, lose **A**.

Cannot keep **C + A** while claiming **P**.



Under partition: **serve stale (AP)** or **refuse / wait (CP)**.

CAP in practice

System	Practical lean	What breaks under partition?
Single-node RDBMS	**C + A**	Partition across nodes barely exists
Cassandra / AP KV stores	**A + P** (tunable C)	May return **stale** values
Spanner / many NewSQL	**C + P**	Latency / temporary **unavailability**
MongoDB	Configurable	Depends on replica-set / write concern

Few systems are “pure CAP letters.” Ask: **when the network splits, do we block or serve stale?**

Keeping strong C: Two-Phase Commit (2PC)

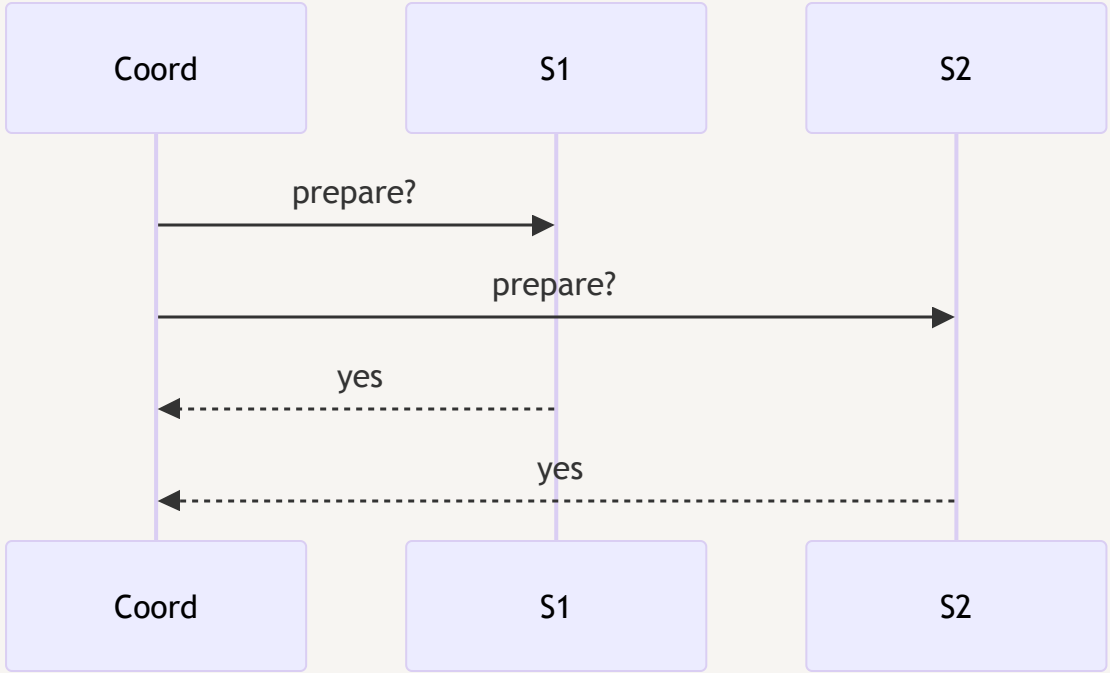
To keep **atomic** multi-node updates (all commit or all abort), classic distributed DBMSs use **Two-Phase Commit** — a **commit protocol**, not Two-Phase **Locking** (2PL).

Phase	Coordinator	Participants
1 · Prepare / vote	Sends *prepare*	Vote **yes** (can commit) or **no**
2 · Commit / abort	If all **yes** → *commit*; else → *abort*	Persist or rollback

Goal: no partial success across nodes. Cost: everyone **waits** on the coordinator and on each other.

2PC — phase 1: vote

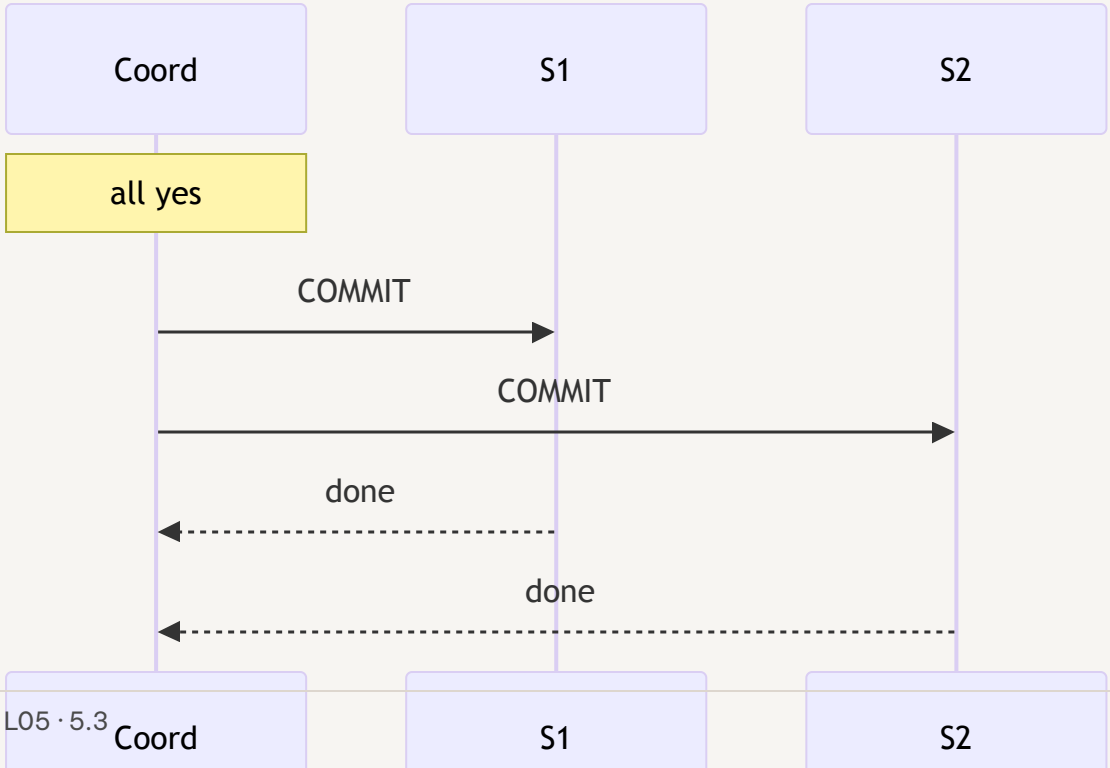
Coordinator: *Can you commit?* → each participant votes **yes** / **no**.



Phase I ends when every vote is in (or timeout).

2PC — phase 2: commit / abort

All **yes** → `COMMIT` everywhere. Any **no** / timeout → `ABORT` everywhere. Slow / partitioned participant **blocks** the rest.



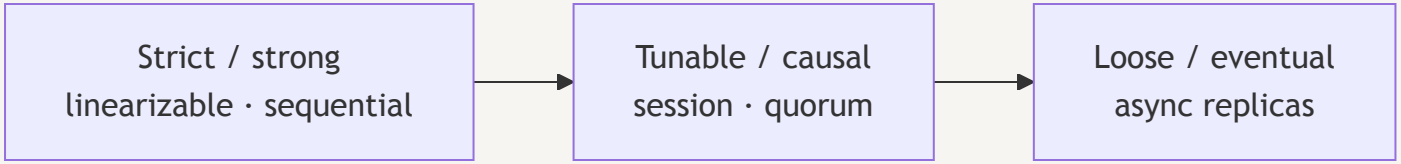
Why many NoSQL systems avoid strict 2PC

If you insist on…	You typically pay…
Cross-node **atomic** commit (2PC-style)	Latency, blocking, coordinator SPOF risk
CP under partition	Temporary unavailability / higher write latency
AP at global scale	Weaker consistency models (next slides)

Large web / IoT stores often choose **availability + partition tolerance**, then **relax** consistency — not because "consistency is useless," but because **2PC does not scale** to thousands of nodes / multi-region paths.

Consistency spectrum

Consistency is not binary. Apps pick a point: **strict enough** for the use case, **loose enough** to scale.



Toward strict	Toward loose
Harder; more coordination / latency	Easier to scale & stay available
Reads see the latest write (more often)	Reads may be **stale** until sync

Good-enough consistency depends on the application — bank ledger ≠ social "like" count.

BASE vs ACID

ACID (classic RDBMS)	BASE (many NoSQL / AP designs)
Atomicity	**B**asically **A**vailable — keep answering
Consistency (strong invariants)	**S**oft state — state may change via background sync
Isolation	**E**ventual consistency — converge when updates stop
Durability	Prefer availability & horizontal scale

BASE is a design slogan, not a formal theorem — it names the AP-leaning trade-off after CAP.

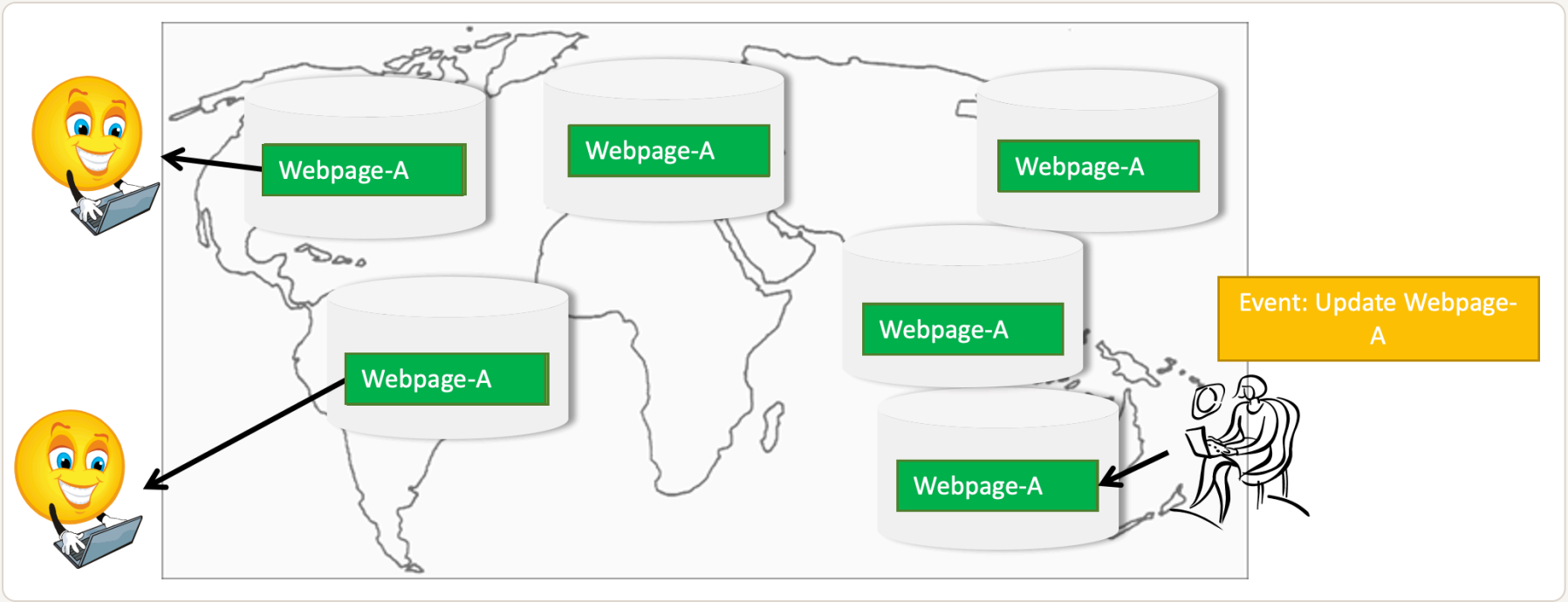
Eventual consistency — definition

Eventual consistency: if no new updates arrive, **all replicas eventually agree** on the same value.

During convergence	After convergence
Reads may be **stale** or disagree across replicas	All clients see the **same** value
Writes still accepted (typical **AP** stance)	System looks “consistent again”

Not “never consistent” — **consistent later**. Latency of convergence depends on replication / conflict resolution.

Eventual consistency — picture



Eventual consistency — web replica story

Update webpage in NA → EU / Asia replicas lag → users abroad may still see the old page → if writes stop, all regions converge to the new page.



Stronger than “eventual”: RYOW

Read-your-own-writes (RYOW): the same client must see its own write even while global replicas lag. Common middle ground (session / strong-read options, e.g. S3).



Consistency — design questions

1. **Profile display name** — need RYOW for that user?
2. **View counter / feed** — is short staleness OK for throughput?
3. **Bank balance / inventory** — need strong / transactional (CP / 2PC-style) semantics?
4. **Multi-region** — latency SLA vs consistency SLA?

Start from the use case → then pick engine + consistency level — not the reverse.

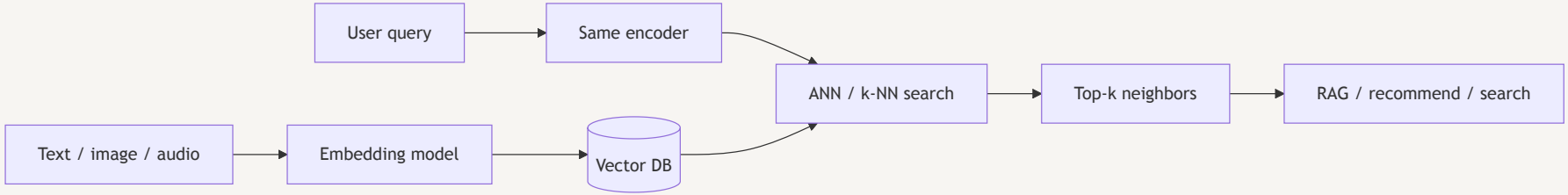
5.4 Vector databases

Why vectors next to NoSQL

Classical KV/document/CF/graph cover structured access. **Vector DBs** add similarity search for RAG / recommend / semantic search — same "specialized store" idea.

SQL-over-stores is §5.5 — not a deep dive into one engine.

Embeddings & similarity search



- Store **dense vectors**; query by **cosine / L2 / inner product**
- Engines: **Pinecone, Milvus, Weaviate, pgvector**, OpenSearch k-NN, …

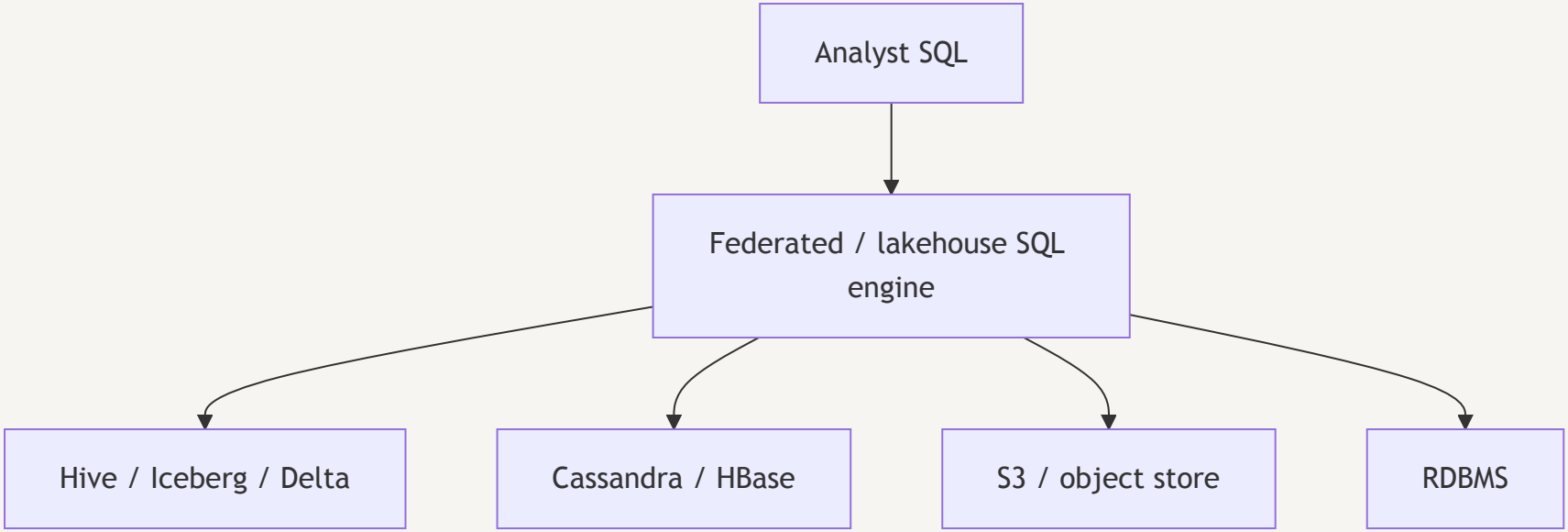
Why vectors matter now

Use case	Role of vector DB
RAG	Retrieve passages for LLM grounding
Semantic search	Meaning > keyword match
Recommendations	Similar users / items
Dedup / clustering	Near-duplicate detection

Part of the 2026 AI data stack — alongside lakes, warehouses, and NoSQL OLTP stores.

5.5 SQL over stores

SQL over heterogeneous stores



- **One SQL dialect**, many connectors — analytics without ETL of everything first
- Complements native NoSQL APIs (key / document / traversal)

Takeaways

1. **5.1** — scale-out + flexible schema pushed specialized engines beside RDBMS
2. **5.2** — KV, document, column-family, graph — choose by access pattern
3. **5.3** — under partition, trade C vs A; eventual + tunable consistency
4. **5.4** — vector search for AI retrieval
5. **5.5** — SQL as a cross-store analytics lens

Next

L06 — DynamoDB